

Convolution Neural Network (CNN)

CNN = Convolution + Neural Network

Neural Network part we know that

Neural Network

- *layers : input ,hidden, output*
- *neurons: how many neurons*
- *activation functions: which activation function which layer*
- *Parameters cal : Gradient descent*
- *You need to think about how to avoid the overfit*
- *You need to think about optimization*
- *How many batches, how many epochs,*
- *which metrics : cross entropy*
- *initializers*

Now we will concentrate on Convolution part

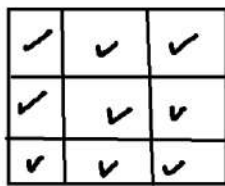
CNN is used for Object detection, Image or Video detection

CNN very famous === So many developments are based on CNN only

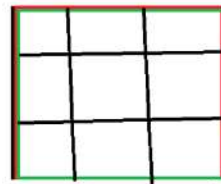
Why CNN

*Imagine we have an image with size 1000 * 1000 (Gray Image)*

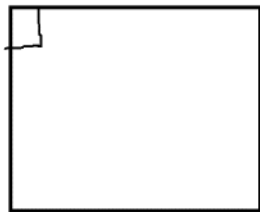
*If it is a color image: **RGB , the image size is : 1000 * 1000 * 3***



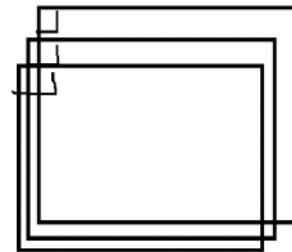
giving $\frac{3 \times 3}{H \times W}$



$3 \times 3 \times 3$ $\textcircled{3} \times \textcircled{3} \times \textcircled{3}$
H W C
27



$1 \times 1 \times 1$



$1 \times 1 \times 1 \times \textcircled{3}$
R G B

- For this shape: $1000 \times 1000 \times 3$, the number of input layer neurons required is $3M$
- after this if you give one hidden layer with 1000 neurons
- How many parameters = $3M \times 1000 + 1000 \approx 3B$
- $3B$ parameters in that starting Input + Hidden layer, to process this we required
- so much big compute engines, practically It is not a good idea

why we need pass entire image , why can't we pass important features

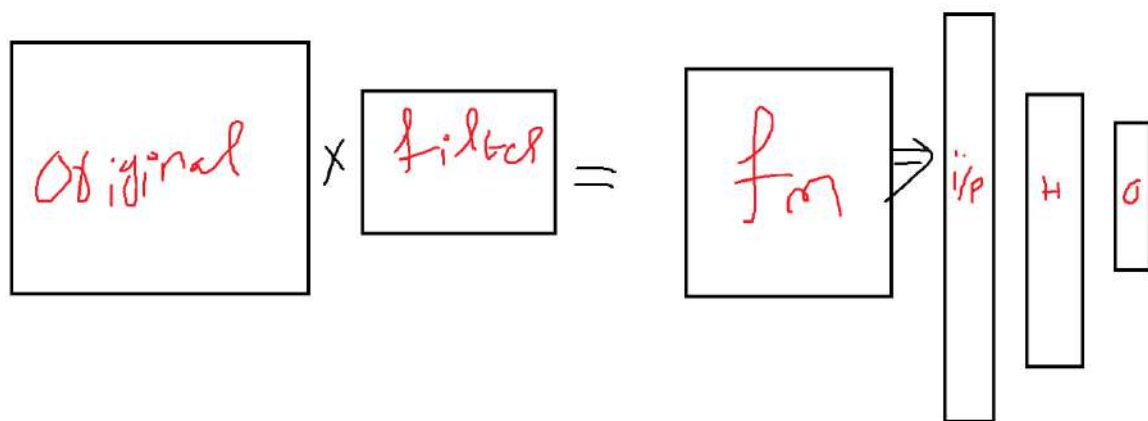
**Idea: Convert Bigger image to small image which is having important features
this image will pass to the Neural network**

Q2) How we get this smaller image from the bigger image

Convolution: Multiplying two Signals =====> produce the another signals

we multiply one image with another image =====> **feature map**

Original image * **filter or kernel** = **feature map**



filters

Already existed kernels or filters

- **sobel filter**: used to detect the vertical or horizontal edges
- **Laplacian filter**: used to detect all types of edges
- **Gaussian blur**: To avoid blurriness and noise

Filters in CNN:

- in CNN filter means weights
- these weights will find during the training by using Back propagation
- good weights leads to good filter leads to good feature map

Q3) How to choose the filter :

In NN filter is nothing but weight matrix

your weight matrix is good, it will multiply with original image === good features

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Input

1	0	1
0	1	0
1	0	1

Filter / Kernel

$$\begin{array}{ccccc}
 & \begin{array}{ccc} 1 \times 1 & 0 \times 1 & 1 \times 1 \end{array} & & \\
 \begin{array}{c} 1 \\ 0 \\ 1 \end{array} & \begin{array}{c} 1 \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 1 \\ 0 \end{array} & \begin{array}{c} 1 \\ 1 \\ 1 \end{array} & \begin{array}{c} 0 \\ 1 \\ 1 \end{array} \\
 \begin{array}{c} 1 \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 1 \\ 0 \end{array} & \begin{array}{c} 1 \\ 1 \\ 1 \end{array} & \begin{array}{c} 0 \\ 1 \\ 1 \end{array} & \begin{array}{c} 0 \\ 0 \\ 1 \end{array} \\
 \begin{array}{c} 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 1 \end{array} & \begin{array}{c} 1 \\ 1 \end{array} & \begin{array}{c} 1 \\ 0 \end{array} & \begin{array}{c} 0 \\ 0 \end{array}
 \end{array}$$

$$1 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 0 \cdot 1 + 1 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 = 4$$

$$\begin{array}{ccccc}
 1 & \begin{array}{c} 1 \\ 0 \\ 1 \end{array} & \begin{array}{c} 0 \\ 1 \\ 0 \end{array} & \begin{array}{c} 1 \\ 1 \\ 1 \end{array} & \begin{array}{c} 0 \\ 0 \\ 1 \end{array} \\
 \begin{array}{c} 0 \\ 0 \\ 0 \end{array} & \begin{array}{c} 1 \\ 0 \\ 0 \end{array} & \begin{array}{c} 1 \\ 1 \\ 1 \end{array} & \begin{array}{c} 0 \\ 1 \\ 1 \end{array} & \begin{array}{c} 0 \\ 0 \\ 1 \end{array} \\
 \begin{array}{c} 0 \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 1 \\ 1 \end{array} & \begin{array}{c} 1 \\ 1 \\ 1 \end{array} & \begin{array}{c} 1 \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 0 \\ 0 \end{array}
 \end{array}$$

$$1 * 1 + 0 * 1 + 1 * 0 + 0 * 1 + 1 * 1 + 0 * 1 + 1 * 0 + 0 * 1 + 1 * 1 = 3$$

First slide to the right side = (4,3,4)

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

Input x Filter

4		

Feature Map

			0	0
0				0
0	0			
0	0			0
0			0	0

First slide to the right side = (4,3,4)

second slide to the right side = (2,4,3)

Third slide to the right side = (2,3,4)

We are multiplying Original image with filter ,we are getting feature map

4	3	4
2	4	3
2	3	4

*Original image size $5 * 5$, Filter size is $3 * 3 =$ Feature map $3 * 3$*

*Original image size $6 * 6$, Filter size is $3 * 3 =$ Feature map $4 * 4$*

*Original image size $7 * 7$, Filter size is $3 * 3 =$ Feature map $5 * 5$*

*Original image size $N * N$, Filter size is $F * F =$ Feature map $N - F + 1 * N - F + 1$*

Color image Convolve with filter

- *color image means 3 chanel RGB*
- *Example gray imag shape : $5 * 5$*
- *Color image shape : $5 * 5 * 3$*
- *they will mention filter size = $4 * 4$*
- *but when we multiply with color image, the filter shape adjusted automatically*
- *so the shape become $4 * 4 * 3$*
- *original image layer multiply with filter layer*
- *the corresponding layer numbers will be add up*

*Original image size $5 * 5 * 3$, Filter size is $3 * 3 * 3 =$ Feature map $3 * 3$*

*Original image size $8 * 8 * 3$, Filter size is $4 * 4 * 3 =$ Feature map $5 * 5$*

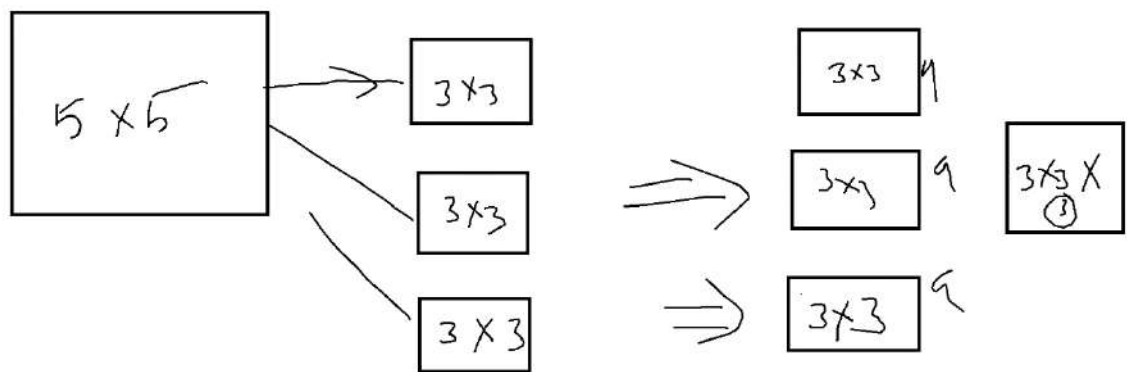
Original image with multiple filters

original image can be multiply with many filters

*$5 * 5 \quad 3 * 3 = 3 * 3$*

*if we have 10 filters the resultant feature map becomes = $773 * 10$*

Type equation here.



Original image size 5×5 , Filter size is $3 \times 3 =$ Feature map 3×3

Filter size is $3 \times 3 =$ Feature map 3×3

Filter size is $3 \times 3 =$ Feature map 3×3

case-1: image gray * filter===== >

8x8 3x3 6x6

case-2: image color * filter === >

8x8x3 3x3x3 6x6

case-3: image(8x8) * f1(3x3) 6x6

* f2(3x3) 6x6 6x6x4

* f3(3x3) 6x6

* f4(3x3) 6x6

case-4: image(8x8x3) * f1(3x3x3) 6x6

* f2(3x3x3) 6x6 6x6x4

* f3(3x3x3) 6x6

* f4(3x3x3) 6x6

Draw back – 1

Original image size $6 * 6$, Filter size is $3 * 3 = FMap: 4 * 4$

Original image size $6 * 6$, Filter size is $4 * 4 = Fmap: 3 * 3$

Original image size $6 * 6$, Filter size is $5 * 5 = Fmap: 2 * 2$

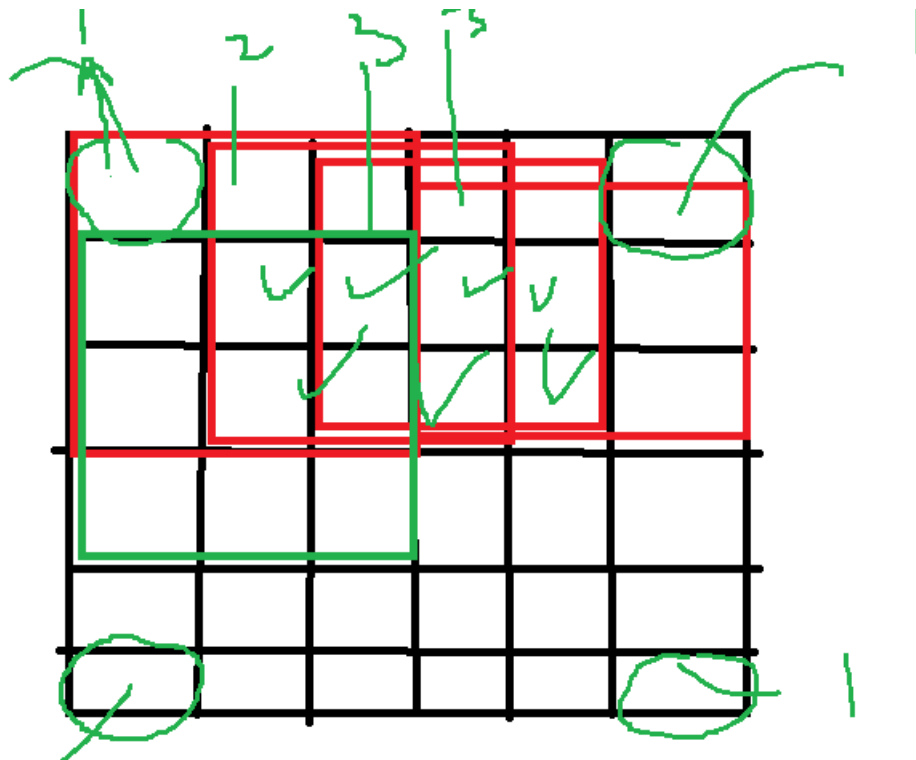
As filter size increases ,resultant image (feature map)size is decreasing

which means number of pixels are decreasing ,Input becomes less

Less input data is not good

CNN: Instead of sending total image,we will multiply a filter and

we will send a small image which is having importnat features



Draw back – 2: while convolution edges are calculated only one time

with this way we might lose the information

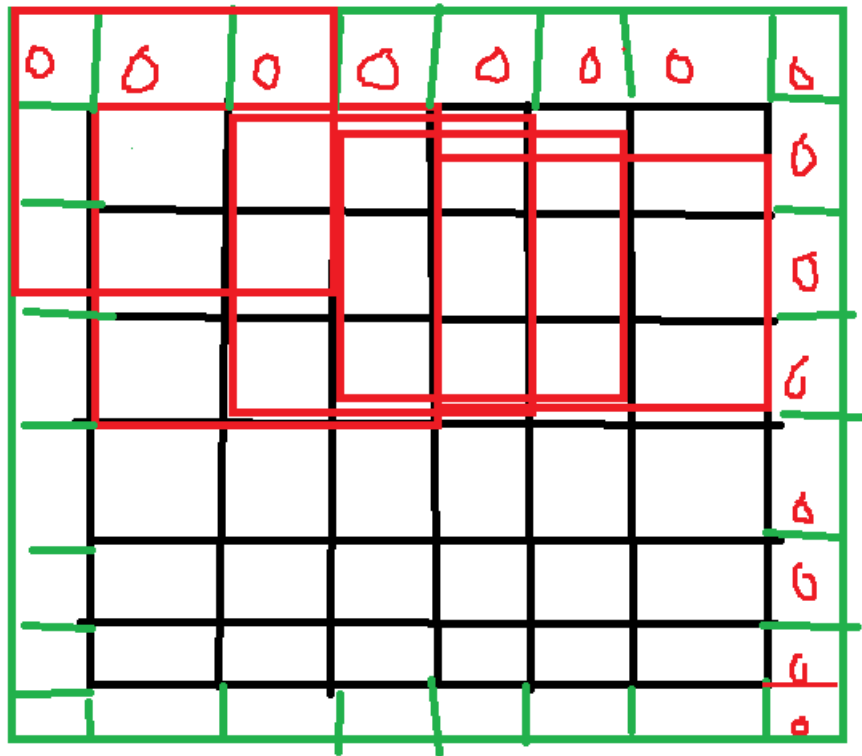
Goal:

we need to avoid image shrink

we need to avoid loss of edges information

Padding: Adding an extra layers

$$(6 \times 6) * (3 \times 3) = 4 * 4$$



padding = 1

adding one extra layer with zeros

padding = 2

adding two extra layers with zeros

*when we add extra layer the original image shape will be increase + 2 * p*

*Original : (6X6) * (3X3) = 4 * 4*

padding = 1

*(6X6) * (3X3) Padding = 1 : 6 * 6*

*6 * 6 becomes 8 * 8 when we apply padding = 1*

*N = 6 * 6 , f = 3 * 3 p = 1 :*

*(6X6) * (3X3) Padding = 2:*

6 * 6 becomes 10 * 10 when we apply padding = 2

$$N + (2p) - F + 1$$

Original Image	Filter	Padding	Feature map
N*N	f*f	No	(N-f+1)*(N-f+1)
N*N	f*f	Yes =P	(N+2p-f+1)*(N+2p-f+1)

Stride:

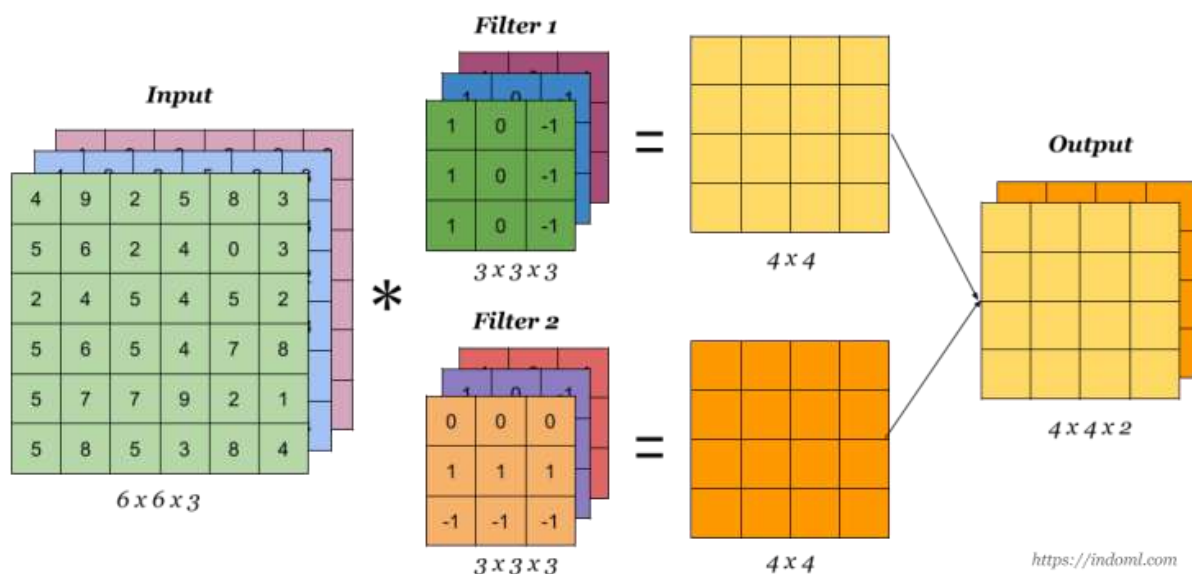
To speed up the computaion process ,the filter convolve with original image
which means sliding will skip by one step

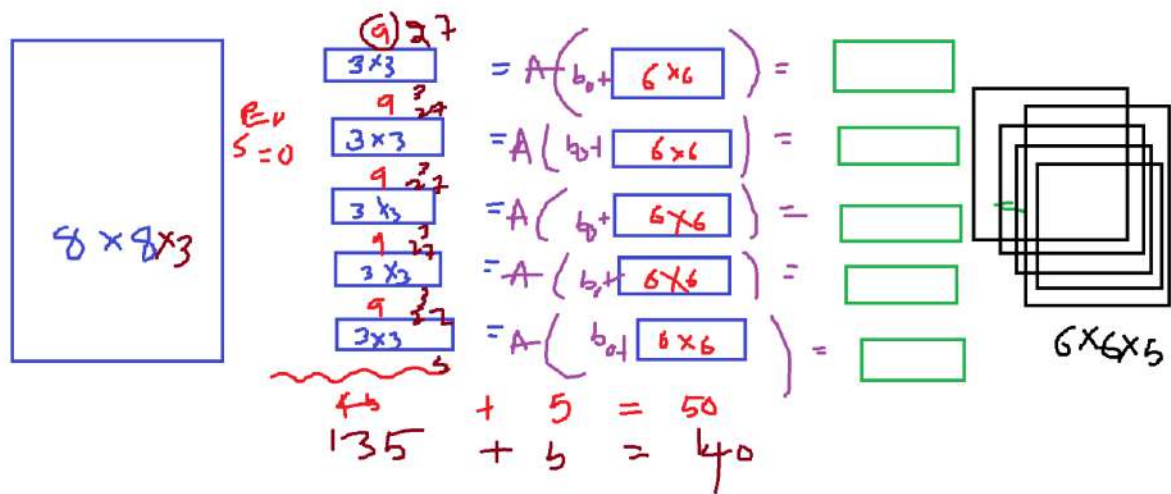
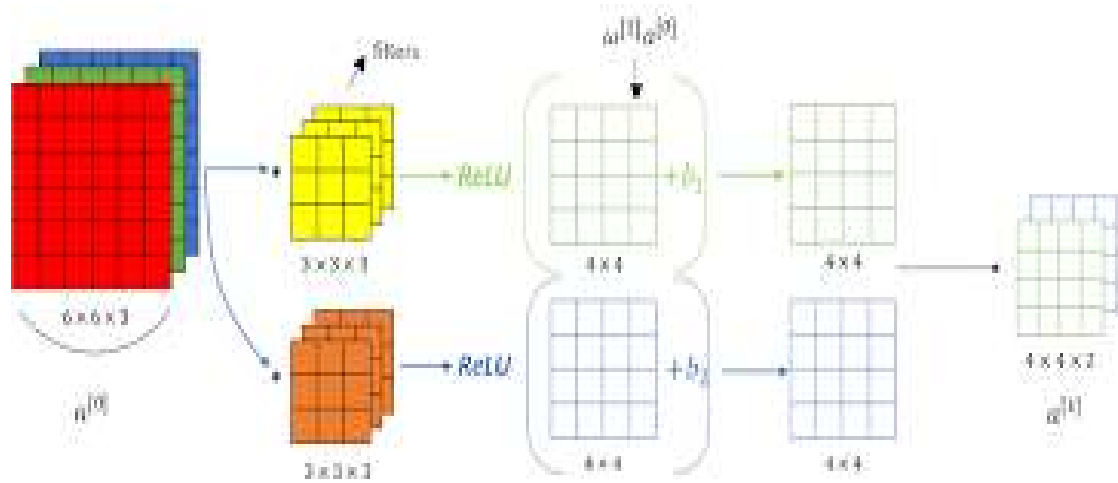
stride is means step size

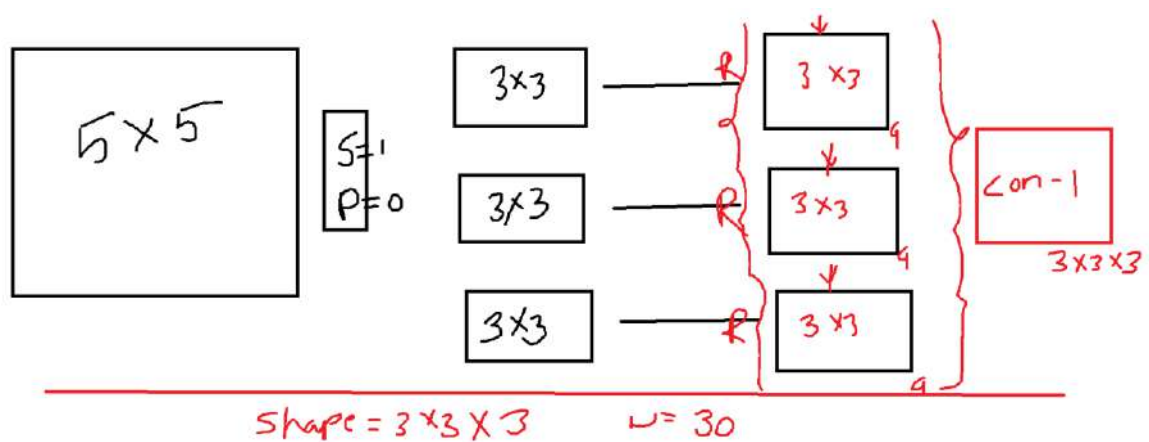
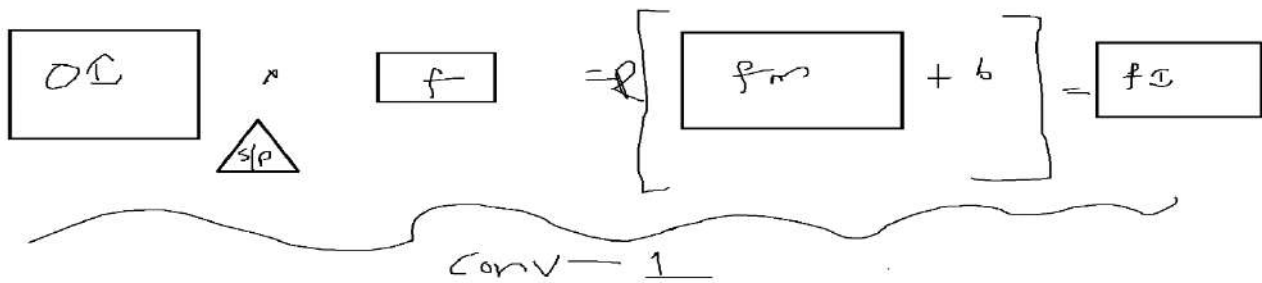
by default stride (s) = 1



Image	Stride	padding	Filter	Feaure map	Formula
8 * 8	1	0	3 * 3	6 * 6	$\frac{n + 2p - f + 1}{s}$
8 * 8	1	1	3 * 3	8 * 8	$\frac{n + 2p - f + 1}{s}$
8 * 8	2	0	3 * 3	3 * 3	$\frac{n + 2p - f + 1}{s}$





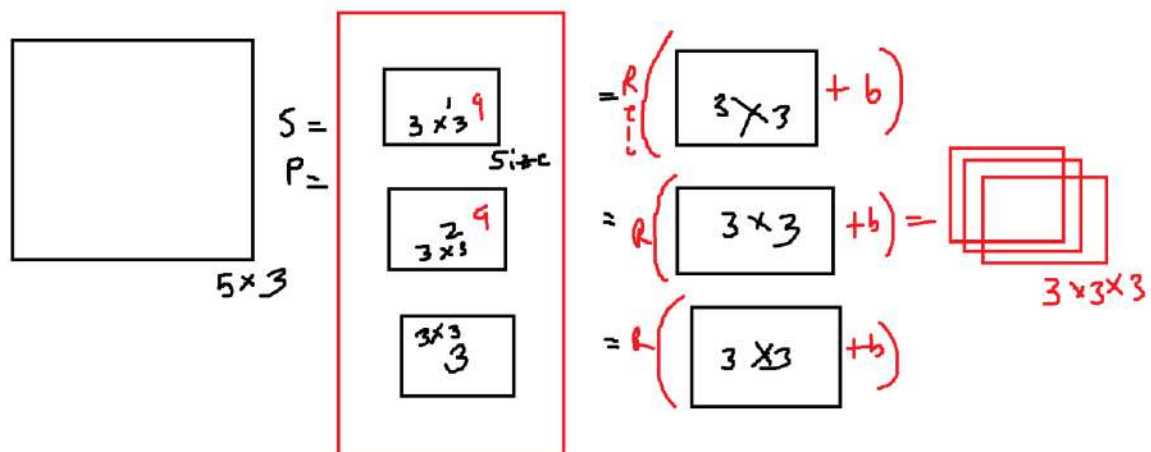


$$(6 * 6) (3 * 3) = (4 * 4)$$

$$(6 * 6 * 3) (3 * 3 * 3) = (4 * 4)$$

Original image is $6 * 6 * 3$, Filter size = $3 * 3 * 3$ Number of filter = 2

Feature map:



Parameters calculation

Assume that we have an image $512 * 512 * 3$ (Color image)

Assume that our filter size = $3 * 3 == > 3 * 3 * 3$

Number of filters = 32

How many parameters = weights + bias

Step – 1: Each filter has $3 * 3 * 3 = 27$ weights

Step – 2: we have 32 filters = $32 * 27 = 864$

Step – 3: When we apply activation function on each feature map

32 filters = 32 bias

Step – 4: Total parameters = $864 + 32 = 896$

Shape:

Image multiply with filter , a feature map will generate it will pass through

Activation function finally we will get output

Number of filters = Number of outputs

In above example

$$(512 * 512 * 3) (3 * 3 * 3) = Relu(510 * 510) = 510 * 510$$

$$(512 * 512 * 3) (3 * 3 * 3) = Relu(510 * 510) = 510 * 510$$

$(512 * 512 * 3) (3 * 3 * 3) = Relu(510 * 510) = 510 * 510$

$32 \text{ times} == > 510 * 510 * 32$

after Conv – 1 layer we will apply pooling layer

Pooling:

Max pool

the image divide by pool size , out of those values maximum will come

Avg pool

the image divide by pool size , avg of all those values will come

*Resultant image = $8 * 8$*

*Pooling size = $2 * 2$*

*Output image size = $4 * 4$*

CNN : Object detection

Models are developed

VGG16

VGG32

mobile net

ResNet

RCNN

FasterRCNN

MASKRCNN

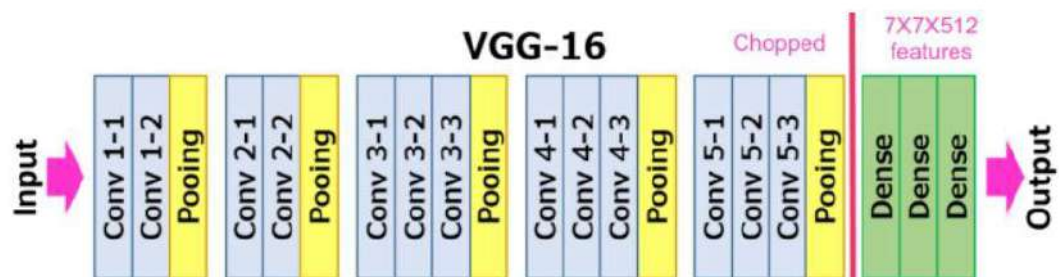
YOLO

In ML model we developed Logistic regression

save the mode === .pkl or .joblib

In DL we trained one model

ANN.KERAS model.save(weights)



$32 * 32$ multiply image with $4 * 4 = 29 * 29$

$32 * 32 * 3$ multiply image with $4 * 4 * 3 = 29 * 29$

$32 * 32$ multiply image with $4 * 4$ (we are using 3 filters) $[4 * 4 * 3] = 29 * 29 * 3$

$32 * 32 * 3$ multiply image with $4 * 4 * 3$ (we are using 3 filters) $[4 * 4 * 3 * 3]$
 $= 29 * 29 * 3$

$32 * 32$ multiply image with $4 * 4$ (we are using 3 filters) $[4 * 4 * 3] = 29 * 29 * 3$

*How many parameters are required? $= 4 * 4 * 3 + 3 = 51$*

*$32 * 32 * 3$ multiply image with $4 * 4 * 3$ (we are using 3 filters) $[4 * 4 * 3 * 3]$
 $= 29 * 29 * 3$*

*How many parameters are required? $48 * 3 + 3 = 147$*

Finally One convolution layer

*Original * Filter ==> Relu(Fmap + bias) = image*

Padding ,stride, Number of filters

Pooling layer:

Ones the image we got, Pooling layers

*some times we feel Covolution result image also have many pixel values
we want to reduce that, from the all pixels we want identify important pixels
imortant pixels having the more information*

- *Average pool*
*For example : $6 * 6$, pooling layer: $2 * 2$*
- *maxpool*

*Convolution – layer1 = $OI * Filter = Relu(Feamap + bias) = Image$*

Inuput: Originalimage ==> Convolution – layer1 + Pooling layer = Final image1

Inuput: Final image 1 ==> Convolution – layer2 + Pooling layer = Final Image2

Inuput: Final image – 2 ==> Convolution – layer3 + Pooling layer = Final Image3

+

*Final Image3: $28 * 28 = 784$ pixels falatten : Fully connected layer*

Hidden layer

Output layer

Convolution Neural Netowrk

CNN is biggest development in DL

in ML we saved the model: Model.pkl file

in DL we can save the model: saving the weights

Our main goal: Find the suitable weights in order to reduce the error

weights file or parameter file

weights depends on model architecture

You can develop one good model, you save the weights

and share that weights to your friend , he will do the prediction === Transfer learning

we are not training the model from scratch, we will use already developed model weights

this is called Transfer learning

CNN ===== so many models

1) LeNet

2) AlexNet: Geffry Hinton GodFather of AI

3) VGG16 , Vgg19, Vgg32

4) MobileNet

5) ResNet 50

6) RCN

7) fastRcnn

8) Mask Rcnn

9) Yolo

CNN python code

MobileNet code

Fine tuning is work

we will use those weights, and we will train the model on our data

we no need to worry about architecture

Directly take that model and pass the input output data

ask the model to learn

